SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	**CSA1205**	**Course Title:**	**Computer Architecture for Multicore Systems**
CO Assessed:	**CO2**	**Assessment:**	**ASSESSMENT TOOL2- DEBUGGING & OPTIMISATION TASK**
Weightage:	**20%**	**Total Marks:**	**25**
CO2 Statement:	*Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)*		
Student Name:	JANNATHUL FIRDAUS N	**Reg.No.:**	192573025
Department:	CSE – DS	**Date:**	

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an

optimized approach using the non-restoring division method, explaining how it improves performance.

5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

*I **JANNATHUL FIRDAUS N (192573025)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	**Debugging Approach & Analysis (20%)**	**Correctness of Debugged Solution (25%)**	**Optimization Techniques Applied (20%)**	**Testing and Documentation (15%)**
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

Q1. Signed Integer Addition Using 2's Complement

A program performing signed integer addition using 2's complement produces incorrect results when adding large values such as **+120 and +50 in 8-bit representation**. The problem can be debugged by examining the binary addition and checking for signed overflow.

Binary Representation

In 8-bit 2's complement representation:

+120

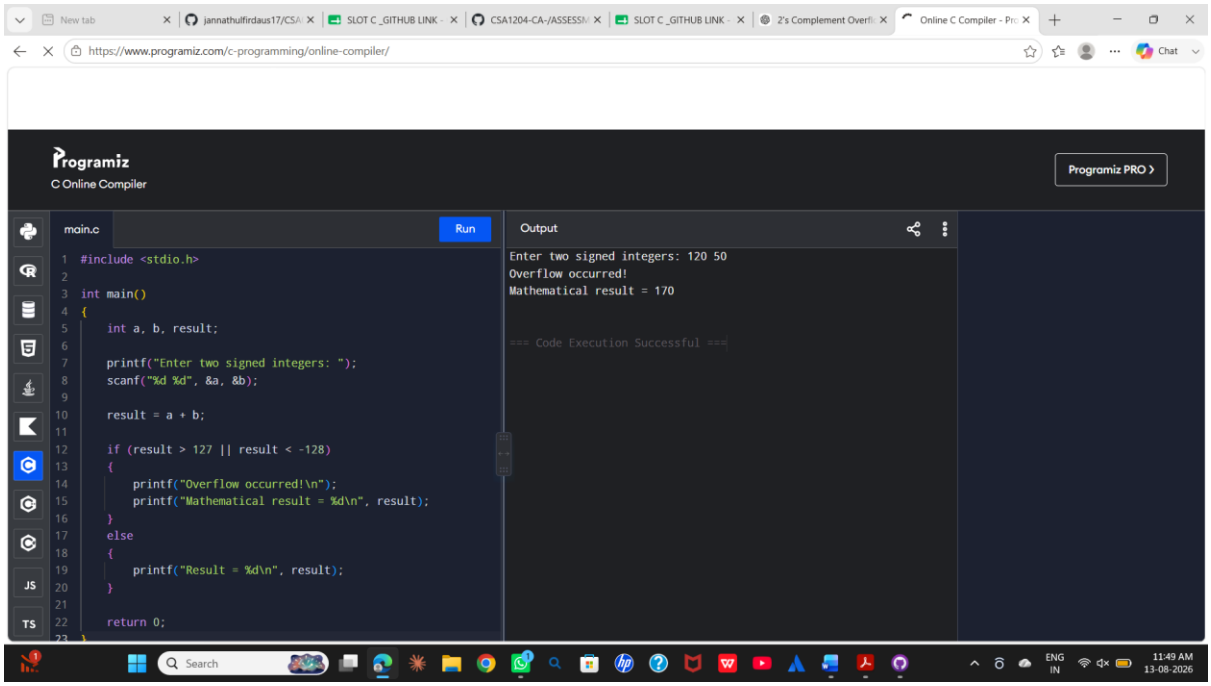
120 = 01111000

+50

50 = 00110010

Now perform binary addition:

```
  01111000
+ 00110010
----------
  10101010
```



Debugging Point

For an 8-bit signed number, the valid range is −128 to +127. If the mathematical result goes outside this range, overflow occurs. The uploaded question specifically uses +120 and +50 as the overflow example.

Optimization

Overflow can also be detected using the sign bits:

- Positive + Positive → Negative = overflow
- Negative + Negative → Positive = overflow

Conclusion

The incorrect result is not caused by an addition error. It is caused by **8-bit signed overflow**. The program should detect the overflow flag and either report the overflow condition or use a larger integer representation.

Q2. Ripple Carry Adder and Carry Look-Ahead Adder

A processor using a **ripple carry adder (RCA)** experiences significant delay during arithmetic operations because the carry must propagate sequentially from one bit position to the next.

Problem in Ripple Carry Adder

For each bit, the carry-out depends on the carry-in from the previous bit.

The carry equation is:

$C_{i+1} = A_iB_i + (A_i \text{ XOR } B_i)C_i$

For an n-bit ripple carry adder, the carry generated at the least significant bit may have to propagate through all n stages before the final result is available.

For example:

$C0 \rightarrow C1 \rightarrow C2 \rightarrow C3 \rightarrow C4 \rightarrow \ldots \rightarrow Cn$

This sequential propagation produces a **carry propagation delay**.

In a real-time system, this delay can reduce arithmetic performance.

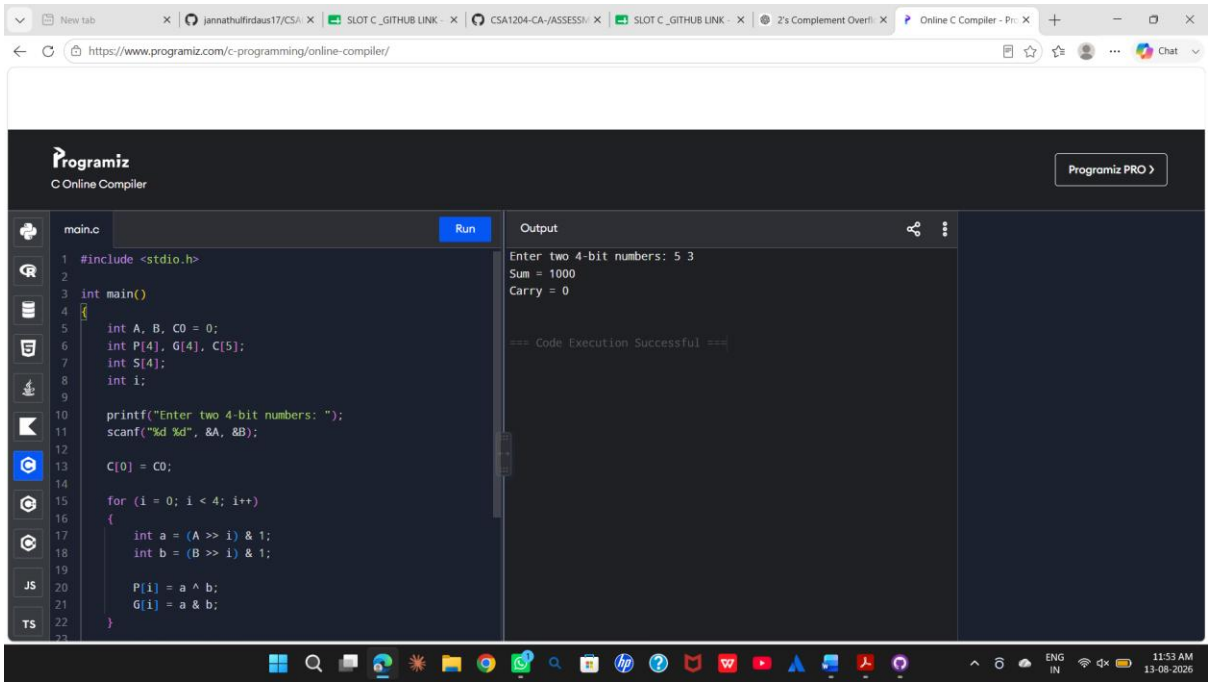
$G_i = A_i \text{ AND } B_i$

$C1 = G0 + P0C0$

$C2 = G1 + P1G0 + P1P0C0$

$C3 = G2 + P2G1 + P2P1G0 + P2P1P0C0$

Thus, the carry signals are generated in parallel.



Advantages

Ripple Carry Adder	Carry Look-Ahead Adder
Carry propagates sequentially	Carry is calculated in advance
Higher propagation delay	Lower propagation delay
Simple hardware	More complex hardware
Suitable for low-speed applications	Suitable for high-speed applications

Conclusion

The RCA is slow because of sequential carry propagation. Replacing it with a **carry look-ahead adder** reduces propagation delay by calculating carries in parallel, making it more suitable for real-time and high-performance processors.

Q3. Debugging Booth's Algorithm for Signed Multiplication

Booth's algorithm is used to perform efficient multiplication of signed binary numbers. Incorrect results for negative operands can occur due to errors in **bit-pair checking, arithmetic shifting, or sign extension**.

Registers Used

Booth's algorithm uses:

- A – Accumulator
- M – Multiplicand
- Q – Multiplier
- Q-1 – Additional bit

Initially:

A = 0

Q = Multiplier

M = Multiplicand

Q-1 = 0

At every iteration, the pair (Q0, Q-1) is examined.

Bit-Pair Decision Table

Q0	Q-1	Operation
0	0	No operation
0	1	A = A + M
1	0	A = A − M
1	1	No operation

After the required addition or subtraction, an **arithmetic right shift** is performed on the combined register (A, Q, Q-1).

Possible Errors

1. Incorrect bit-pair checking

The algorithm must examine the current Q0 and Q-1 bits correctly. If the pair is incorrectly interpreted, the wrong addition or subtraction operation will be performed.

2. Logical shift instead of arithmetic shift

For negative operands, a logical right shift inserts zero into the most significant bit. This destroys the sign information.

An arithmetic right shift must preserve the sign bit.

For example:

Negative number:

10110010

Arithmetic right shift:

11011001

The leading 1 is retained.

3. Incorrect sign extension

Negative multiplicands must be represented using 2's complement and sign-extended to the required register width.

4. Incorrect subtraction

For the (10) bit pair:

$A = A - M$

Subtraction must be performed correctly using 2's complement arithmetic.

5. Incorrect number of iterations

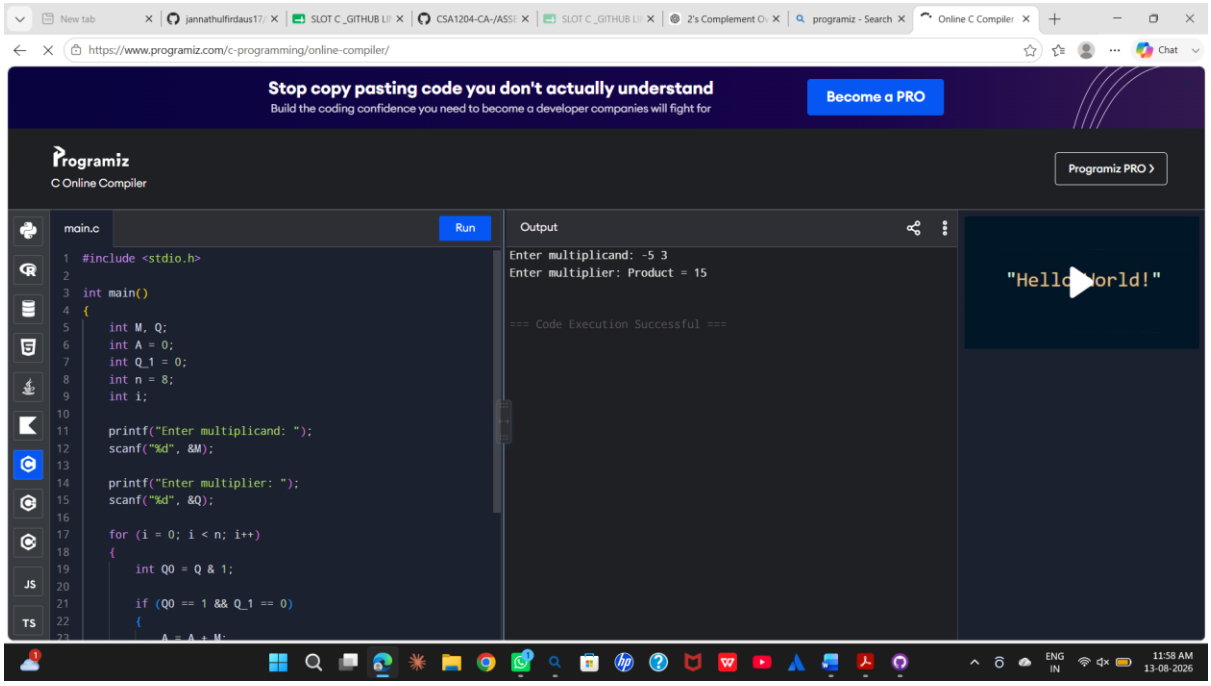
For n-bit operands, Booth's algorithm should perform n iterations.

Debugging and Correction

The implementation should follow these steps:

1. Initialize A = 0.
2. Load the multiplier into Q.
3. Set Q-1 = 0.
4. Represent negative operands using 2's complement.
5. Examine (Q0, Q-1).
6. Perform the appropriate addition or subtraction.
7. Perform an arithmetic right shift.
8. Preserve the sign bit during shifting.
9. Repeat for the required number of iterations.

10. Combine A and Q to obtain the final product.



Conclusion

Incorrect signed multiplication in Booth's algorithm is commonly caused by improper bit-pair checking, incorrect arithmetic shifting, or loss of sign information. Correct 2's complement representation, sign extension, arithmetic shifting, and iteration control ensure accurate signed multiplication.

Q4. Restoring Division and Non-Restoring Division

The **restoring division algorithm** can become inefficient because it repeatedly restores the partial remainder whenever a subtraction produces a negative result.

Restoring Division Process

The basic operation is:

$A = A - M$

where:

- A = partial remainder
- M = divisor

If the result is negative, the divisor must be added back:

If $A < 0$:

$A = A + M$

This is called the **restoration step**.

The workflow is:

Shift

↓

Subtract divisor

↓

Check remainder

↓

Negative?

/ \

Yes No

↓ ↓

Restore Continue

Source of Inefficiency

The main inefficiency occurs when:

$A = A - M$

produces a negative remainder.

The algorithm then performs:

$A = A + M$

to restore the previous value.

Therefore, two arithmetic operations may be required for a single unsuccessful subtraction.

Repeated restoration increases execution time.

Optimized Solution: Non-Restoring Division

The **non-restoring division algorithm** avoids immediately restoring a negative remainder.

Instead, the sign of the current remainder determines the next operation.

If the remainder is positive:

$A = 2A - M$

If the remainder is negative:

$A = 2A + M$

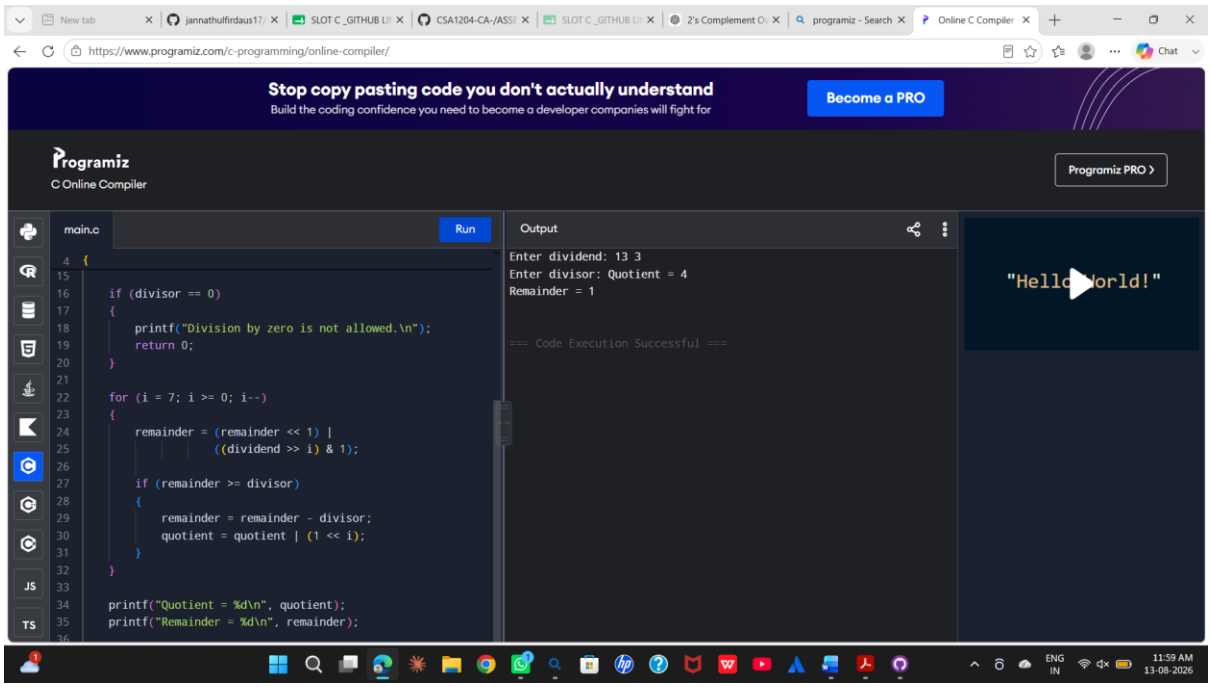
Thus, the algorithm does not repeatedly subtract and restore the divisor.

Comparison

Restoring Division	**Non-Restoring Division**
Negative remainder is restored immediately	Negative remainder is retained temporarily
Requires restoration operations	Avoids repeated restoration

Restoring Division	Non-Restoring Division
More arithmetic operations	Fewer arithmetic operations
Higher execution time	Improved execution time
Simple implementation	More efficient implementation

At the end of the non-restoring algorithm, if the remainder is negative, a final correction is performed.



Conclusion

The major inefficiency in restoring division is the repeated restoration of negative partial remainders. **Non-restoring division** improves performance by avoiding these repeated restoration operations and performing a correction only when necessary.

Q5. IEEE 754 Single-Precision Floating-Point Addition

IEEE 754 single-precision floating-point representation uses **32 bits**:

1 bit → Sign

8 bits → Exponent

23 bits → Fraction

The problem occurs when adding a very small number to a very large number. The main causes of inaccurate results are **exponent alignment, limited precision, normalization, and rounding**.

Exponent Alignment

Before two floating-point numbers are added, their exponents must be made equal.

For example:

Large number = 1.000000×10^{20}

Small number = 1.000000×10^{0}

The smaller number must be shifted to align its exponent with the larger number.

Conceptually:

1.000000×10^{20}

$0.000000... \times 10^{20}$

Because single precision has limited significant bits, many bits of the smaller number can be lost during this alignment.

As a result, the smaller number may have little or no effect on the final result.

Normalization

After addition, the result must be normalized into the standard form:

1.xxxxx × 2^E

If the addition produces an extra leading bit, the significand is shifted and the exponent is increased.

If necessary, the significand is shifted in the opposite direction and the exponent is decreased.

Rounding Error

Single precision has limited precision. When more bits are produced than can be stored in the 23-bit fraction field, the extra bits must be rounded.

IEEE 754 uses rounding rules to determine the final stored value.

Therefore, rounding can introduce a small difference between the exact mathematical result and the stored floating-point result.

Example

Consider:

$1.0 \times 10^{20} + 1.0$

The value 1.0 is extremely small compared with 1.0×10^{20}.

During exponent alignment, the significand of 1.0 must be shifted by a very large amount. Its significant bits can fall outside the available precision.

Consequently, the final floating-point result may effectively remain:

$\approx 1.0 \times 10^{20}$

even though mathematically the result is slightly larger.

Optimization Techniques

1. Use double precision

Double precision provides more fraction bits and therefore greater numerical precision.

2. Use appropriate summation algorithms

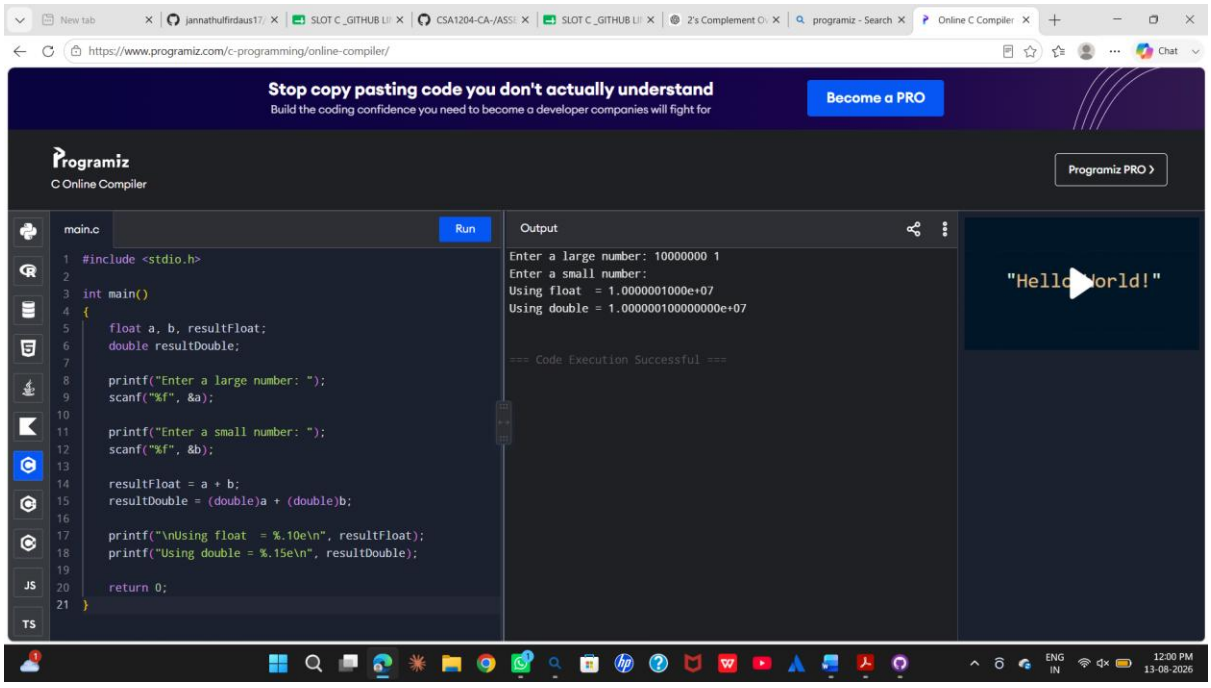
For repeated floating-point additions, techniques such as compensated summation can reduce accumulated rounding errors.

3. Avoid adding numbers with extremely different magnitudes whenever possible

Values can be grouped or processed in a way that reduces loss of significance.

4. Preserve intermediate precision

Using a higher-precision representation during intermediate calculations can reduce rounding errors.



Conclusion

The inaccurate result is primarily caused by the limited precision of IEEE 754 single precision. During exponent alignment, the smaller operand can lose significant bits. Normalization and

rounding can introduce additional errors. Using **double precision and suitable numerical algorithms** can significantly improve the accuracy of floating-point calculations.